Multi-Source Candidate Data Transformer — Technical Design (Stage 1)

Problem. Ingest candidate data from many heterogeneous, overlapping, often malformed sources and emit **one canonical profile per candidate** — fixed fields, normalized formats, deduplicated, with provenance and confidence. Core principle: **wrong-but-confident is worse than honestly-empty** → never invent values.

Pipeline

```
detect/route → source adapters → normalize → merge & resolve → confidence → CANONICAL RECORD (internal, full schema) →
projection (runtime config) → validate → output
```

The load-bearing decision: a **hard separation between the internal canonical record and the output projection**. The engine always produces the full canonical record; a runtime config only reshapes a *view*. Same engine, no code changes.

Canonical schema & normalized formats

Fixed fields: `candidate_id, full_name, emails[], phones[], location{city,region, country}, links{linkedin,github,portfolio,other[]}, headline, years_experience, skills[{name,confidence,sources[]}], experience[{company,title,start,end,summary}], education[{institution,degree,field,end_year}], provenance[{field,source,method}], overall_confidence`.

Formats: phones → **E.164**; dates → **`YYYY-MM`** (or `YYYY` — we never invent a month); country → **ISO-3166 alpha-2**; skills → **canonical names** (alias map + conservative fuzzy match); emails lowercased + deduped. Normalizers are pure and total: garbage in → `None` out, never an exception.

Merge / conflict resolution & confidence

- **Match key:** shared normalized email, else shared normalized full name (union-find). Different names are *not* fuzzily merged (documented limit).
- **Scalar winner:** `source_reliability_weight × method_certainty` (ATS, CSV > GitHub > notes), deterministic tie-break. **All** contributing sources — winners and losers — are recorded in `provenance`.
- **Lists** (emails/phones/skills): union + dedupe; skills corroborated by more sources rank higher.
- **Confidence:** fixed weight tables, +bonus for agreement, ×penalty for conflict, clamped to [0,1]. No randomness/time/network → identical inputs, identical numbers. `overall_confidence` = mean of populated per-field confidences.

Runtime custom-output config (the twist)

A config selects a subset of fields, remaps from a canonical path (`from`, e.g. `emails[0]`, `skills[].name`, `location.country`), re-normalizes per field, toggles confidence/provenance, and sets `on_missing` = `null | omit | error`. A JSON Schema is **derived from the config** (types + `required`) and every projected view is validated before return. The default schema validates the default output. Projection + validation live in their own layer, fully decoupled from the engine.

Edge cases (and the decision for each)

1. **Conflicting scalars** (CSV "Acme" vs ATS "Acme Corp", differing titles) → weighted winner; all values retained in provenance; confidence reduced.
2. **Garbage** (`phone="not-a-number"`, `date="sometime 2021"`, invalid JSON file) → value dropped / source skipped; the run never crashes.
3. **Skill aliasing** (`reactjs`/`React.js`/`react` → `React`) → merged; confidence rises with corroboration count.
4. **Missing required under `on_missing:"error"`** → structured per-candidate error, isolated so a batch of thousands still completes.
5. **Open-ended role** (`to: present`) → `end = null`.

Trust layer (beyond spec — our differentiator)

`provenance` and `confidence` are given as fields but nobody says what to *do* with them. The resolver retains **every** proposal (winners *and* losers), and a trust layer turns that into: (1) a field-level **why** audit trail (chosen value, the alternatives it beat, conflict + reason); (2) a per-candidate **data-quality / conflict report** (completeness %, disagreeing sources with competing values, flags) plus a batch rollup; (3) **review-queue gating** that routes low-confidence / incomplete profiles to `needs_review` with explicit reasons — so a wrong-but-confident profile is flagged, never trusted silently. Reuses data already computed; no extra passes.

Scope deliberately deferred (under time pressure)

LinkedIn (no public API / ToS); ML/LLM-based identity resolution and résumé extraction — replaced by deterministic heuristics + a fuzzy-matched alias map. Five sources — Recruiter CSV + ATS JSON (structured) and GitHub API + recruiter notes + résumés (PDF/DOCX, unstructured) — exercise the full merge/conflict path.

Surface: a minimal Flask web UI (primary) plus a thin CLI; both call one shared engine. **Determinism with a live GitHub call:** the single HTTP seam is mocked in tests and the committed sample outputs are generated with `--no-github`.